

内容保真度与治理质量评估智能体 —— 研发计划书

课题：大模型内容安全与质量评估 / 数据治理前后文本比对

技术栈：Python + DeepSeek (LLM-as-Judge) + LangChain + Streamlit

制定日期：2026-06-23

研发周期：2026-06-26 ~ 2026-07-10 (10 个工作日)

版本：v1.0

目录

- 项目概述
- 已有成果物
- 研发任务分解 (6 个 Phase)
- 两周工作计划 (6.26 ~ 7.10)
- 风险分析与应对预案
- 计划调整机制

1. 项目概述

1.1 目标

构建 **LLM-as-Judge** 智能体，自动比对"治理前原始文本"与"治理后重写文本"，从两个维度审计治理质量：

维度	说明	权重
语义一致性	核心事实、实体、量化指标是否被误改或丢失	0.5
可读性与结构质量	排版、语句通顺度、表格对齐	0.5

1.2 核心架构

用户输入 (治理前/后文本) → 锚点预处理 ([Before N] / [After N] 标尺)
→ Prompt 构建 (System Prompt 约束 + User Template 拼接)
→ DeepSeek API (ChatOpenAI, temperature=0.0)
→ JSON 解析与清洗 (去 think 标签 + 异常兜底)
→ Streamlit 可视化 (Metric 卡片 / DataFrame 瑕疵表 / Pass-Fail 徽章)

1.3 验收标准

编号	指标项	目标值
I1	与人工一致性 (Pearson r)	≥ 0.8
I2	评分稳定性 (方差)	< 0.005
I3	瑕疵检出 F1	≥ 0.8

编号	指标项	目标值
I4	锚点定位准确率	≥ 90%
I5	可解释性（每维度含 reason 字段）	已实现
I6	瑕疵可定位（行级锚点）	已实现
I7	可复现（temperature=0.0 + SHA256 令牌）	已实现
I8	抗偏置（长度/位置偏置检测）	已实现

2. 已有成果物

模块	文件	状态
核心评判引擎	app/engine.py	
数据模型（Pydantic）	app/models.py	
Prompt 模板	app/prompts.py	
抗偏置模块	app/debias.py	
指标计算	app/metrics.py	
一致性校准	app/calibration.py	
稳定性分析	app/stability.py	
综合报告生成	app/reporter.py	
FastAPI 路由	app/routes.py + main.py	
中文前端页面	static/index.html	
Streamlit 演示应用	app.py	
测试用例	tests/test_evaluate.py	
评估数据集	data/eval_dataset.json	

3. 研发任务分解（6 个 Phase）

Phase 1：数据模型与核心引擎

步骤	任务内容	交付物
P1-1	定义 Pydantic 数据模型（AnchorSpan / FlawItem / DimensionScore / EvalRequest / EvalResponse）	app/models.py
P1-2	设计 System Prompt（双维度评分细则 + 瑕疵类型/严重程度约束 + JSON Schema）	app/prompts.py

步骤	任务内容	交付物
P1-3	封装 DeepSeek LLM 调用（ChatOpenAI 单例，temperature=0.0）	app/engine.py
P1-4	实现 JSON 解析器（兼容 Markdown 代码块、think 标签清洗、大括号定位）	app/engine.py
P1-5	实现锚点预处理算法 <code>build_anchored_text()</code> （按行标记 [Before N]/[After N]）	app.py
P1-6	基础冒烟测试（LLM 调用 + JSON 解析端到端验证）	tests/

Phase 2：抗偏置与稳定性保障

步骤	任务内容	交付物
P2-1	长度偏置检测（治理前后长度比 + 风险等级 High/Medium/Low）	app/debias.py
P2-2	位置偏置检测（瑕疵在全文分布位置分析，检测 front_bias/back_bias）	app/debias.py
P2-3	偏置缓解得分（维度标准差 × 0.5 + 长度因子 × 0.5）	app/debias.py
P2-4	抗偏置 Prompt 补充指令（长度无关/位置无关/格式无关/领域无关）	app/debias.py
P2-5	评分稳定性验证（多次采样 n≥3，方差 < 0.005）	app/stability.py
P2-6	可复现性令牌（SHA256 哈希，固定策略下结果一致）	app/engine.py

Phase 3：评估指标与校准

步骤	任务内容	交付物
P3-1	瑕疵检出指标（Precision / Recall / F1）	app/metrics.py
P3-2	锚点定位准确率（字符容差 ≤ 10 字符匹配）	app/metrics.py
P3-3	一致性校准（Pearson r / Spearman ρ / MAE / RMSE / 一致率）	app/calibration.py
P3-4	构建/扩充评估数据集（含人工标注 Ground Truth）	data/eval_dataset.json
P3-5	Prompt 调优迭代（多轮校准，目标 Pearson r ≥ 0.8）	测试报告

Phase 4：报告生成与批量评估

步骤	任务内容	交付物
P4-1	综合评估报告生成器（7 大模块集成：评估/校准/稳定性/瑕疵/锚点/偏置/可复现）	app/reporter.py
P4-2	JSON 报告导出	app/reporter.py

步骤	任务内容	交付物
P4-3	控制台摘要输出（ANSI 彩色格式化）	app/reporter.py
P4-4	验收标准检查清单自动判定	app/reporter.py
P4-5	批量评估脚本（遍历 eval_dataset.json）	main.py

Phase 5：Web 应用与可视化

步骤	任务内容	交付物
P5-1	FastAPI 后端（评估 API 端点）	app/routes.py
P5-2	中文前端页面（HTML/CSS/JS 静态文件）	static/index.html
P5-3	Streamlit 演示版（侧边栏 API Key + 双文本域 + 实时评估）	app.py
P5-4	结果可视化（Metric 卡片 / DataFrame 瑕疵表 + 严重程度着色 / Pass-Fail 徽章）	app.py
P5-5	加载动画（st.spinner）+ 异常友好提示	app.py
P5-6	自定义 CSS（渐变标题 / 圆角卡片 / 按钮 hover 动效）	app.py

Phase 6：测试、调优与答辩准备

步骤	任务内容	交付物
P6-1	端到端集成测试（API → LLM → 解析 → 渲染）	测试报告
P6-2	边界测试（空文本 / 超长文本 / 特殊字符 / 中英混合）	测试用例
P6-3	异常测试（API Key 错误 / 网络超时 / JSON 畸形 → 不白屏）	测试用例
P6-4	答辩 PPT（项目背景 / 架构 / 核心技术 / 指标数据 / Demo 截图）	PPT 文件
P6-5	演示脚本（含 3 组典型用例：优秀治理 / 过度清洗 / 严重误改）	演讲稿
P6-6	最终验收 + 材料打包提交	全部材料

4. 两周工作计划（6.26 ~ 7.10）

基于已有成果物做增量完善，聚焦测试、校准、调优与答辩准备。每日 6h 有效工时。

日期	星期	核心任务	对应 Phase	关键交付物
6.26	四	校准数据准备 + 首次校准测试	P3	eval_dataset.json + 初始 Pearson r
6.27	五	Prompt 调优 + 多轮校准验证	P3	Pearson r $\geq$ 0.8
6.28	六	偏置分析集成 + 长度/位置偏置报告	P2	偏置分析通过
6.29	日	稳定性验证 + 可复现性测试	P2	方差 < 0.005
6.30	一	瑕疵检出指标 (F1) + 锚点定位准确率	P3	F1 $\geq$ 0.8, 准确率 $\geq$ 90%
7.1	二	端到端集成测试 + 边界/异常 case	P6	集成测试报告
7.2	三	批量评估跑通 + 综合报告生成	P4	批量评估 + JSON 报告
7.3	四	UI 精修 + CSS 美化 + 体验优化	P5	最终版 Streamlit 页面
7.4	五	答辩 PPT + 系统架构图 + 流程图	P6	PPT 初稿
7.5	六	答辩演讲稿 + 3 组 Demo 用例准备 + 彩排	P6	演示脚本终稿
7.6	日	缓冲日 (补漏 / 提前完成可休息)	—	—
7.7	一	UI 细节打磨 + 性能优化	P5/P6	优化版 app.py
7.8	二	全指标复核 + Bug 修复	P6	验收指标汇总表
7.9	三	PPT 终稿 + 材料整理打包	P6	全部材料就绪
7.10	四	最终提交 / 推送 GitHub	P6	提交确认

里程碑节点

里程碑	完成标志	目标日期
M1: 校准达标	Pearson r $\geq$ 0.8	6.27
M2: 指标全部达标	F1 / 锚点 / 偏置 / 稳定性 通过	6.30
M3: 集成测试通过	全流程无 Bug	7.1
M4: 演示材料完备	PPT + 脚本 + Demo 就绪	7.5
M5: 提交	全部材料打包提交	7.10

进度落后应对

落后天数	措施
1 天	利用 7.6 (缓冲日) 补齐
2 天	合并 PPT 与演讲稿制作, 精简非核心页面

落后天数

措施

3 天+

立即与老师沟通，优先保证 Demo 可演示

5. 风险分析与应对预案

风险	概率	影响	预案
DeepSeek API 宕机	中	高	本地缓存 5-10 组评估结果用于演示
LLM 输出 JSON 格式不稳定	中	高	多层解析兜底（代码块提取 → 大括号定位 → retry）；严格 Prompt 约束
人工标注数据不足	中	高	用 AI 生成 mock 数据，人工抽查校验后使用
评分与人工一致性不达标	中	高	2-3 轮 Prompt 调优；引入 few-shot 示例
网络不可达 GitHub/DeepSeek	中	中	配置代理；本地离线开发，代码本地 Git 管理
答辩时间提前	低	高	提前 3 天完成核心功能，PPT 先行
电脑故障/环境丢失	低	高	每日提交 Git + 环境 requirements.txt 完整

6. 计划调整机制

触发条件	措施
进度超前 ≥ 1 天	后续任务提前，考虑优化/加分项
进度延迟 ≥ 1 天	分析瓶颈，调整次日计划，必要时启用降级方案
关键阻塞 1 天内未解	求助老师/同学，使用备选方案
验收标准变更	重新评估工时，优先保障核心指标

文档维护：随研发进度动态更新。每个里程碑完成后同步更新完成状态与日期。